

- Final Client Round — Banking Domain Questionnaire (Part 3)
 - Section 5: Scenario-Based Questions (5 Questions)
 - Q26. It's 2 AM and your core banking application's payment processing latency has spiked from 200ms to 5 seconds. Walk me through your incident response.
 - Q27. The bank's AWS bill has increased 40% month-over-month. How do you investigate and optimize?
 - Q28. A security audit finds that an IAM user has had AdministratorAccess for 18 months with no MFA. How do you remediate?
 - Q29. The bank is running a legacy monolith on EC2. How would you approach modernization to microservices on EKS?
 - Q30. How would you handle a scenario where a Terraform apply partially fails in production?
 - Section 6: Data Platform & Observability (5 Questions)
 - Q31. How would you design a data lake architecture for a bank's regulatory reporting needs?
 - Q32. How do you design an observability strategy for a banking platform on AWS?
 - Q33. How do you manage log retention and analysis for banking compliance?
 - Section 7: Leadership & Communication (5 Questions)
 - Q34. Describe a time you had to make a critical architectural decision under pressure. What was your approach?
 - Q35. How do you communicate a production outage to non-technical banking stakeholders?
 - Q36. How do you evaluate and recommend AWS services when the bank's security team is risk-averse about adopting new services?
 - Q37. How would you handle a situation where a development team wants to deploy directly to production, bypassing the CI/CD pipeline?
 - Q38. How do you stay current with AWS services and bring innovation to a conservative banking environment?
 - Quick Reference — Banking Acronyms
 - Interview Closing Tips

Final Client Round — Banking Domain Questionnaire (Part 3)

Candidate: Pushparaj Naik — AWS Cloud Architect (22+ years)

Context: Final round with banking-sector client

Focus: Scenarios, Data Platform, Observability, Leadership

Section 5: Scenario-Based Questions (5 Questions)

Q26. It's 2 AM and your core banking application's payment processing latency has spiked from 200ms to 5 seconds. Walk me through your incident response.

Answer:

Immediate (0-5 minutes):

1. Acknowledge the PagerDuty/SNS alert and join the war room bridge
2. Check **CloudWatch dashboard** — identify which tier is slow:
 - ALB target response time → is it application or downstream?
 - RDS **ReadLatency/WriteLatency** → database bottleneck?
 - EKS pod CPU/memory → compute saturation?
 - ElastiCache hit rate → cache miss storm?
3. Check **recent deployments** — was anything deployed in the last 2 hours? If yes, that's the prime suspect

Triage (5-15 minutes): 4. If **database**: Check **DatabaseConnections**, **CPUUtilization**, active queries

- Lock contention? → Kill blocking query, file ticket for query optimization
 - Connection exhaustion? → Restart connection pooler (PgBouncer/RDS Proxy)
5. If **application**: Check pod logs (**kubectl logs -f**), look for:

- Timeout errors to downstream services
- Memory pressure / OOM kills
- Thread pool exhaustion

6. If **external dependency**: Check third-party API response times, circuit breaker state

Mitigate (15-30 minutes): 7. **Quick wins:** Scale up pods (`kubectl scale`), failover to read replica, clear cache 8. If deployment-related: **Rollback immediately** — `kubectl rollout undo` 9. If data-related: Switch traffic to DR if primary is unrecoverable

Post-Incident: 10. Write blameless post-mortem within 48 hours 11. Update runbook with new scenario 12. Create monitoring for the root cause that wasn't previously covered 13. Present findings to the CAB (Change Advisory Board)

From my experience: At HP/Wipro, I managed production environments supporting QA and performance testing cycles. The key lesson: always check recent changes first — 80% of incidents correlate with a recent deployment or configuration change.

Q27. The bank's AWS bill has increased 40% month-over-month. How do you investigate and optimize?

Answer:

Investigation (Day 1):

1. **AWS Cost Explorer** — break down by:

- Service (which service grew?)
- Account (which account?)
- Region (unexpected region usage?)
- Tag (which application/team?)
- Usage type (data transfer? compute hours? storage?)

2. **Common culprits in banking:**

Cause	Detection	Fix
Forgot to terminate dev/test resources	Untagged resources in Cost Explorer	Tag enforcement + auto-shutdown Lambda
NAT Gateway data transfer	VPC Flow Logs analysis	VPC endpoints for S3/DynamoDB/ECR
Over-provisioned RDS	CloudWatch CPU Utilization < 20%	Right-size or switch to Aurora Serverless
EBS snapshots accumulating	Snapshot age > 90 days	Lifecycle policy with DLM
CloudWatch Logs ingestion	Excessive debug logging in prod	Reduce log level, sampling
Cross-AZ data transfer	ALB logs showing cross-AZ traffic	Zone-aware routing, topology hints

Optimization Plan:

1. Quick Wins (Week 1):

- Right-size EC2/RDS using AWS Compute Optimizer recommendations
- Delete unused EBS volumes and old snapshots
- VPC endpoints for S3 and DynamoDB (eliminates NAT Gateway charges)
- Switch dev environments to Spot instances or auto-shutdown after hours

2. Medium Term (Month 1):

- Reserved Instances or Savings Plans for stable workloads (1-year, no upfront for banking flexibility)
- Aurora Serverless v2 for variable-load databases
- S3 Intelligent-Tiering for data lakes
- Graviton instances for EKS nodes (20% cheaper, better performance)

3. Long Term (Quarter 1):

- Implement FinOps practice with showback/chargeback to business units
- AWS Budgets with alerts at 80% threshold per account
- Monthly cost review with architecture team

Banking-specific note: Never optimize security for cost. KMS keys (1/month), *CloudTrail* (0 for management events), GuardDuty (~\$4/account/month) are non-negotiable. The cost of a breach far exceeds these.

Q28. A security audit finds that an IAM user has had **AdministratorAccess** for 18 months with no MFA. How do you remediate?

Answer:

This is a **P1 security incident** in banking. Immediate response:

Step 1: Contain (within 1 hour)

```
# Deactivate access keys immediately
aws iam update-access-key --user-name affected-user --access-key-id AKIA...
--status Inactive

# Remove console access
aws iam delete-login-profile --user-name affected-user

# Detach AdministratorAccess
aws iam detach-user-policy --user-name affected-user \
--policy-arn arn:aws:iam::aws:policy/AdministratorAccess
```

Step 2: Investigate (24-48 hours)

- Pull CloudTrail logs for the user's entire 18-month history
- Athena query:

```
SELECT eventTime, eventName, sourceIPAddress, userAgent, errorCode
FROM cloudtrail_logs
WHERE userIdentity.arn LIKE '%affected-user%'
AND eventTime > '2024-11-01'
ORDER BY eventTime DESC
```

- Look for: suspicious IP addresses, unusual API calls, data exfiltration patterns
- Check if access keys were ever exposed (GitHub commit scanning, Secrets Hub)

Step 3: Remediate (Week 1)

- Replace IAM user with SSO federated access (IAM Identity Center)
- Implement MFA requirement via SCP:

```
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "BoolIfExists": { "aws:MultiFactorAuthPresent": "false" }
  }
}
```

- Deploy AWS Config rule: **iam-user-mfa-enabled** with auto-remediation
- Deploy AWS Config rule: **iam-policy-no-statements-with-admin-access**

Step 4: Report

- File compliance incident report
- Notify CISO and compliance team
- Update security baseline and SCPs to prevent recurrence
- Add to quarterly audit review

Q29. The bank is running a legacy monolith on EC2. How would you approach modernization to microservices on EKS?

Answer:

I follow the **Strangler Fig pattern** — incrementally extract services while the monolith continues running.

Phase 1: Assess & Plan (4-6 weeks)

- Map the monolith's domain boundaries (payments, accounts, customers, notifications)
- Identify candidate services to extract first (choose low-risk, well-bounded domains)
- Define API contracts between services
- Set up EKS cluster alongside existing EC2 infrastructure

Phase 2: Strangler Fig Extraction (6-12 months)

Phase 2a: Route traffic through API Gateway

Client → API Gateway → Monolith (all traffic)

Phase 2b: Extract first service (e.g., notifications)

Client → API Gateway → Notification Service (EKS)
→ Monolith (everything else)

Phase 2c: Extract more services iteratively

Client → API Gateway → Notification Service (EKS)
→ Customer Service (EKS)
→ Payment Service (EKS)
→ Monolith (shrinking)

Phase 2d: Monolith fully decomposed

Client → API Gateway → All Microservices (EKS)

Key Banking Considerations:

- **Database per service:** Each microservice gets its own database (avoid shared DB)
- **Event-driven integration:** Use EventBridge/SQS for loose coupling between services
- **Transaction management:** Implement Saga pattern for distributed transactions (no 2PC)
- **Data consistency:** Accept eventual consistency for non-financial reads; strong consistency for writes
- **Dual-write period:** During migration, both monolith and new service may need to write to same data — use CDC (Change Data Capture) to keep in sync

From my Advantest project, I led containerization of GLP and FNO using ECS — the same decomposition approach applies but with EKS for more complex banking microservices.

Q30. How would you handle a scenario where a Terraform apply partially fails in production?

Answer:

Partial failures are dangerous because infrastructure is in an **inconsistent state**.

Immediate Response:

1. **Don't panic, don't re-run blindly.** Read the error message carefully.
2. Check **terraform state list** — see what was created vs. what failed
3. Check the AWS console — verify the actual state of resources

Common Scenarios:

Scenario	Response
Resource created but not all tags applied	Fix the tag issue, re-run terraform apply — it's idempotent
Timeout creating RDS (takes 15+ min)	Increase timeout in provider config, re-run apply
IAM eventually consistent issue	Wait 30 seconds, re-run apply
Dependency failure (resource B failed because A didn't finish)	Re-run apply — Terraform retries failed resources
State lock stuck (previous run crashed)	terraform force-unlock <LOCK_ID> after verifying no concurrent run
Resource created in AWS but not in state	terraform import to bring it under management

Prevention:

- Use **-target** for risky changes to apply in stages
- Always run **plan** before **apply**
- Use **lifecycle { prevent_destroy = true }** on critical resources (RDS, S3 with data)
- Break large changes into smaller PRs

Banking-specific: Always have a rollback plan documented in the change request before applying to production.

Section 6: Data Platform & Observability (5 Questions)

Q31. How would you design a data lake architecture for a bank's regulatory reporting needs?

Answer:

Medallion Architecture (Bronze/Silver/Gold):

Data Sources → Bronze (raw) → Silver (cleaned) → Gold (business-ready)

Bronze Layer (S3):

```
s3://bank-datalake/bronze/
├── core-banking/transactions/dt=2026-05-21/
├── cards/settlements/dt=2026-05-21/
├── loans/disbursements/dt=2026-05-21/
└── Format: JSON/CSV (as received from source systems)
```

Silver Layer (S3):

```
s3://bank-datalake/silver/
├── transactions_cleaned/ (deduplicated, validated, PII masked)
├── settlements_enriched/ (joined with reference data)
└── Format: Parquet (columnar, compressed, partitioned)
```

Gold Layer (S3):

```
s3://bank-datalake/gold/
├── daily_transaction_summary/
├── regulatory_ctr_report/ (Currency Transaction Report)
├── suspicious_activity_flags/ (SAR data)
└── Format: Parquet (aggregated, ready for BI tools)
```

Processing Pipeline:

```
Sources → Glue Crawlers → Glue Catalog → Glue ETL Jobs → Silver/Gold
                                         → Athena (ad-hoc queries)
                                         → QuickSight (dashboards)
                                         → Redshift (heavy analytics)
```

This is exactly the pattern I implemented at Rio Tinto for the Databricks-ready data platform — S3 data stores with Glue jobs and standardized templates.

Q32. How do you design an observability strategy for a banking platform on AWS?

Answer:

Three Pillars + Banking Extensions:

Pillar	Tool	What We Capture
Metrics	CloudWatch + Prometheus (EKS)	Latency, error rate, throughput, saturation
Logs	CloudWatch Logs + OpenSearch	Application logs, audit logs, access logs
Traces	X-Ray + OpenTelemetry	Request flow across microservices
Security	GuardDuty + Security Hub	Threat detection, compliance posture
Business	Custom CloudWatch metrics	Transaction success rate, payment volume, SLA compliance

Key Banking Metrics (Golden Signals):

1. Transaction Success Rate	→ Target: > 99.95%
2. Payment Processing Latency	→ Target: P99 < 500ms
3. Failed Authentication Rate	→ Alert: > 5 per minute (possible brute force)
4. Database Connection Pool	→ Alert: > 80% utilization
5. Certificate Expiry	→ Alert: < 30 days remaining

Dashboard Hierarchy:

- **Executive:** Transaction volume, revenue, SLA compliance, cost
- **Operations:** Service health, error rates, deployment status, incident count
- **Security:** Failed logins, policy violations, GuardDuty findings, Config compliance %
- **Engineering:** Pod metrics, database performance, API latency percentiles

From my experience at Wipro/HP, I configured CloudWatch + SNS for monitoring and used Datadog for EC2 observability — the banking layer adds business metrics and compliance dashboards.

Q33. How do you manage log retention and analysis for banking compliance?

Answer:

Retention Requirements:

Log Type	Minimum Retention	Storage Tier
CloudTrail (API audit)	7 years	S3 → IA → Glacier
Application logs	1 year (active), 7 years (archive)	CloudWatch → S3
VPC Flow Logs	1 year	S3
ALB Access Logs	1 year	S3
Database audit logs	7 years	S3 (from RDS)
WAF logs	1 year	S3 via Kinesis Firehose

Architecture:

All Logs → CloudWatch Logs (hot, 30 days)

- Subscription Filter → Kinesis Firehose → S3 (warm, 1 year)
- S3 Lifecycle → Glacier (cold, 7 years)
- S3 Object Lock (COMPLIANCE mode – immutable)

Analysis:

- **Real-time:** CloudWatch Logs Insights for operational debugging
- **Ad-hoc investigation:** Athena queries over S3 log data (fraud investigation, audit response)
- **Security analytics:** OpenSearch for pattern detection, alerting on anomalies
- **Compliance reporting:** Automated weekly/monthly reports via Lambda + QuickSight

Section 7: Leadership & Communication

(5 Questions)

Q34. Describe a time you had to make a critical architectural decision under pressure. What was your approach?

Answer:

Situation: At ITC Infotech (Rio Tinto project), we needed to decide between Azure Synapse (existing) and AWS Databricks for the data platform migration. The project timeline was tight (3 months), budget was fixed, and stakeholders were split.

Approach:

1. **Gathered data, not opinions:** Ran performance benchmarks with actual Rio Tinto workloads on both platforms
2. **Documented trade-offs as an ADR** (Architecture Decision Record):
 - AWS Databricks: Better ML integration, team already had skills, but higher cost
 - Azure Synapse: Already licensed, but limited ML capabilities and vendor lock-in
3. **Presented to stakeholders** with quantified comparison (cost, performance, migration effort, operational overhead)
4. **Decision:** AWS-hosted Databricks-ready platform with Terraform IaC — balanced cost, capability, and team skills

Outcome: Delivered the platform on time with modular Terraform patterns that enabled the team to add new data sources (S3, RDS, Glue) quickly while maintaining production readiness.

Key lesson for banking: In banking, speed matters but so does auditability.

Documenting the decision with an ADR ensures regulators can understand why a particular technology was chosen.

Q35. How do you communicate a production outage to non-technical banking stakeholders?

Answer:

Template I use:

INCIDENT NOTIFICATION – [Severity Level]

WHAT HAPPENED:

Internet banking login is experiencing slower than normal response times. Some customers may see timeout errors.

WHO IS AFFECTED:

~15% of internet banking users attempting login between 2:00–2:30 AM

BUSINESS IMPACT:

Estimated 2,000 failed login attempts during the window.
No financial transactions were affected. No data loss occurred.

WHAT WE'RE DOING:

The on-call team has identified the root cause (database connection pool exhaustion) and is implementing the fix.
Expected resolution: 30 minutes.

WHAT WE'VE DONE TO PREVENT RECURRENCE:

1. Increased connection pool limits
2. Added automated scaling for peak load
3. Added monitoring alarm for connection pool at 70% (early warning)

NEXT UPDATE: In 30 minutes or when resolved (whichever comes first)

Key principles:

- **Lead with impact**, not technical details
- **Avoid jargon** — "database connection pool exhaustion" becomes "the system couldn't handle the number of simultaneous users"
- **Always include** what you're doing to fix it AND prevent recurrence
- **Set expectations** for next update

Q36. How do you evaluate and recommend AWS services when the bank's security team is risk-averse about adopting new services?

Answer:

My approach is evidence-based, not opinion-based:

1. Map to compliance frameworks:

- "Is this service PCI-DSS compliant?" → Check AWS PCI-DSS compliance page
- "Is it SOC 2 Type II audited?" → Check AWS SOC reports
- "Does it support KMS encryption?" → Verify in documentation

2. Proof of Concept in sandbox:

- Deploy the service in a sandbox account with realistic (but synthetic) data
- Run security scanning tools against it
- Document security controls available (encryption, access control, logging)

3. Present as risk-vs-reward:

- Risk: New service, less internal expertise
- Reward: 60% cost reduction, 3x performance, native compliance features
- Mitigation: Training plan, vendor support, gradual adoption (pilot → rollout)

4. Reference banking peers:

- "Top 5 US banks use this service" (publicly available AWS case studies)
- AWS Financial Services competency partners' reference architectures

5. Document in ADR and get sign-off from security, compliance, and architecture teams

Q37. How would you handle a situation where a development team wants to deploy directly to production, bypassing the CI/CD pipeline?

Answer:

Short answer: No. This is non-negotiable in banking.

How I enforce it:

1. Technical controls (prevention):

- SCPs deny `ecs:UpdateService`, `eks:*`, `lambda:UpdateFunctionCode` for human IAM roles
- Only the CI/CD pipeline's IAM role can deploy to production
- Production Kubernetes RBAC: developers have read-only access

2. Process controls (governance):

- Change management policy: all production changes require approved change request
- Emergency change process exists for P1 incidents — requires CISO/VP approval, retrospective within 48 hours

3. Cultural approach:

- Understand WHY they want to bypass — is the pipeline too slow? Fix the pipeline.
- If deployments take 2 hours, reduce to 15 minutes with parallel testing, cached builds, pre-built images
- Make the right thing the easy thing

Q38. How do you stay current with AWS services and bring innovation to a conservative banking environment?

Answer:

1. **Continuous Learning:** AWS re:Invent sessions, AWS blogs, Well-Architected reviews, hands-on labs
2. **Certifications:** I hold AWS Solutions Architect Associate and GCP Professional Cloud Architect — cross-cloud perspective helps evaluate objectively
3. **Proof of Concept culture:** For every new service I recommend, I build a working PoC with cost analysis, security review, and comparison with current approach
4. **Innovation within guardrails:** I don't propose bleeding-edge services for core banking — I propose mature services (GA for 1+ year, PCI-compliant, SOC 2 audited) with clear banking use cases
5. **Community:** Participate in AWS User Groups, contribute to internal tech talks, mentor junior architects

Example: At ITC Infotech, I introduced GitHub Actions with OIDC for CI/CD — replacing long-lived AWS credentials. This was innovative (security improvement) but within banking guardrails (OIDC is a mature standard, no credentials stored).

Quick Reference — Banking Acronyms

Acronym	Full Form	Relevance
PCI-DSS	Payment Card Industry Data Security Standard	Card payment processing compliance
SOX	Sarbanes-Oxley Act	Financial reporting controls
RBI	Reserve Bank of India	Indian banking regulator
GDPR	General Data Protection Regulation	EU data privacy (if global bank)
SOC 2	Service Organization Control Type 2	AWS service audit reports
RPO	Recovery Point Objective	Acceptable data loss
RTO	Recovery Time Objective	Acceptable downtime
CDE	Cardholder Data Environment	PCI-DSS scope boundary
CAB	Change Advisory Board	Change management approval body
SAR	Suspicious Activity Report	Anti-money laundering reporting
CTR	Currency Transaction Report	Regulatory transaction reporting
BCP	Business Continuity Plan	Disaster recovery planning
IRSA	IAM Roles for Service Accounts	EKS pod-level AWS access
TES	Token Exchange Service	IoT Greengrass credential management
SCP	Service Control Policy	AWS Organizations guardrails

Interview Closing Tips

When the interviewer asks "Do you have any questions for us?":

1. "What are the biggest architectural challenges you're currently facing with your AWS infrastructure?"
 2. "How does your team handle change management for production deployments today?"
 3. "What's the current state of your infrastructure-as-code adoption? Are you looking to standardize on Terraform?"
 4. "How do you balance innovation with the regulatory requirements of banking?"
 5. "What does success look like for this role in the first 6 months?"
-

Good luck, Pushparaj! 🚀

Your 22 years of experience, AWS + GCP certifications, and hands-on project work (Rio Tinto Databricks migration, Advantest ECS containerization, HP K-GPT, Nokia NetAct) give you a strong foundation. The key for the banking client round is to **always connect your answers back to security, compliance, and auditability** — that's what banking clients care about most.